개인 회고

진행하면서 막혔던 부분과 해결

막힌 부분	원인	해결
Inpainting 모델이 로드되지 않음 (흐름 4·5)	원본의 `stabilityai/stable-diffusion-2-inpainting` 저장소가 Hugging Face에서 내려가 `401 Repository Not Found`가 났다. 원본 코드는 `try/except`가 에러를 잡아서, 에러 없이 이미지만 안 나오고 끝났다	같은 모델을 올려 둔 `sd2-community/stable-diffusion-2-inpainting`으로 주소만 바꿨다. 원본 코드 중 바꾼 곳은 이 줄뿐이다
다시 실행한 흐름 4의 그림 모양이 달라짐	처음 실행 때 흐름 5가 `display_image`를 다른 버전으로 다시 정의해 두었다. 노트북은 셀 순서가 아니라 실행 순서대로 변수가 바뀐다	세션을 다시 시작하고 처음부터 순서대로 전체 실행했다

실습을 통해 알게 된 것

Stable Diffusion은 무작위 노이즈에서 시작해서, 프롬프트에 맞게 노이즈를 조금씩 걷어 내며 그림을 만든다. 512×512 픽셀을 바로 다루지 않고 64×64로 줄인 잠재 공간에서 계산해서, L4 GPU에서 30스텝 한 장이 2.2초면 됐다.

Inpainting은 사진과 마스크를 같이 넣으면 마스크의 흰 부분만 다시 그린다. 회색 판의 원 안에만 결함 자국이 생기고 바깥은 그대로였다. 정상 제품 사진에 결함 위치를 칠하면 부족한 불량 데이터를 늘릴 수 있겠다는 걸 알게 됐다.

교수님이 말씀하신 "loss만 보고 판단한 실험은 실패"라는 말이 생성 모델에도 그대로 해당된다는 것을 확인했다. 숫자 하나만 보고 판단하면 안 된다. guidance를 바꿔 가며 CLIP 점수(그림이 프롬프트와 얼마나 맞는지)를 재 보니, 색이 깨지고 뭉개진 guidance 20이 가장 높은 점수(35.45)를 받았다. CLIP 점수는 내용만 보고 품질은 보지 않기 때문이다. 그림을 같이 보지 않았다면 20이 가장 좋다고 착각했을 것이다.

개인 실험 요약

교수님이 말씀하신 비교 설계와 믿을 수 있는 수치를 생각해서, AI가 제안한 실험 중 세 가지를 골라 해 봤습니다. 원본 코드는 건드리지 않았고, 실험 셀에는 모두 `[개인 실험]` 표시를 했습니다. 세 실험 모두 시드를 고정해서 비교하려는 값만 바뀌게 했습니다.

1. **스텝 수**: 원본의 30스텝이 적당한지 궁금해서 5·10·20·30·50·100스텝으로 그려 봤습니다. 5·10스텝은 흐릿했고, 20스텝부터 구도가 정해진 뒤에는 국기 같은 작은 부분만 바뀌었습니다. 시간은 스텝 수에 비례해(30스텝 2.20초, 100스텝 6.76초) 20~30스텝이 적당하다고 생각했습니다. 노이즈가 많은 앞쪽 스텝에서 큰 구도가 정해지고, 뒤쪽 스텝에서 세부가 다듬어지기 때문입니다.
2. **guidance_scale**: guidance를 1·3·5·7.5·8.5·12·20으로 바꿔 시드 3개로 CLIP 점수를 쟀습니다. 1은 연필 그림처럼 나왔고 20은 색이 깨졌으며, 3~12는 점수 차이가 시드에 따른 흔들림보다 작아 어느

값이 낫다고 할 수 없었습니다. guidance가 1이면 부정 프롬프트가 아예 빠지고, 너무 크면 예측값을 지나치게 키워 학습 범위를 벗어나기 때문입니다.

3. **말이 빠진 이유**: 흐름 3 그림에 말이 없어서, 우연인지 확인하려고 시드 20개로 그리고 물체 탐지 모델(Faster R-CNN)로 말이 있는지 판정했습니다. 우주인은 20장 모두 나왔지만 말은 10장만 나왔고, 말이 없는 그림은 대부분 달 표면이었습니다. 그림 한 장으로 모델을 판단하면 안 되고, 시드를 고정해 여러 장으로 봐야 한다는 걸 알게 됐습니다.

어려웠던 것

원본 모델 저장소가 사라졌는데도 `try/except` 때문에 에러 없이 끝나서, 출력 문구를 읽기 전까지는 무엇이 잘못됐는지 몰랐다. 예외 처리가 있는 코드는 결과가 정말 나왔는지 출력까지 확인해야 한다.

좋았던 점

L4 GPU에서 한 장이 2초 정도라, 설정을 바꿔 가며 여러 장을 그리는 실험을 금방 해 볼 수 있었다.

현장에 쓴다면

정상 제품 사진에 결함을 넣을 위치를 마스크로 칠하고 Inpainting으로 불량 이미지를 만들어, 부족한 불량 데이터를 늘리는 데 쓸 것 같다. 다만 실험 ③처럼 원하는 그림이 절반만 나올 수 있다. 생성은 장당 몇 초면 되지만, 그만큼 더 만들고 쓸 만한 그림을 사람이 골라내는 비용이 든다. 그래서 시드와 설정을 고정해 결과를 재현할 수 있게 하고, 골라내는 비용까지 따져서 쓸지 정할 것이다.

앞으로 해 볼 것

6장 실습의 VisA PCB 정상 사진에 Inpainting으로 결함을 그려 넣고, PaDiM이 그 사진을 불량으로 잡아내는지 확인해 보고 싶다. 생성한 결함이 실제 불량 데이터 대신 쓸 만큼 그럴듯한지 확인하는 방법이 될 것 같다.